CASE STUDY #2: INTEGRATIVE PROGRAMMING AND TECHNOLOGY STACK - GROUPINGS
LEARNING OUTCOMES:
By completing this case study, students will be able to:

- Design structured XML documents
- Validate XML using DTD and XSD
- Extract data using XPath
- Transform XML using XSLT
- Explore advanced XML technologies (XQuery & Assertions)

"Smart Academic Records Management System (SARMS)"
Background
Pamantasan ng Lungsod ng Pasig (PLP) plans to modernize its academic records through a **Smart Academic Records Management System (SARMS)**. The system will store, validate, transform, and query structured academic data using XML technologies.
The IT team (your group) is tasked to design a **prototype XML-based system** that ensures:

- Store structured academic data using **XML**
- Ensure data integrity via **DTD and XSD**
- Transform data into readable formats using **XSLT**
- Extract and filter data using **XPath**
- Support advanced querying (**XQuery – self-study**)
- Enforce rules using **XSD Assertions – self-study**

General Tasks:
Each group will design and implement a **mini XML-based system INTEGRATING SCENARIOS GIVEN PER GROUP** that includes:

1. **XML Document**
2. **DTD (structure validation)**
3. **XSD (advanced validation)**
4. **XPath Queries**
5. **XSLT Transformation (XML → HTML Report)**

Each group is assigned **ONE scenario/module**, but:

- All outputs must be **integrated into a single XML file (SARMS.xml)**
- Each module becomes a **section of the system**

Additionally (Self-Study): (refer to the attached supplemental module)

- Apply **XQuery**
- Use **XSD Assertions** for rule validation

Group Composition

- 5 Members per group
- Suggested roles:
  - XML Designer
  - Schema Validator (DTD/XSD)
  - XPath Specialist
  - XSLT Developer
  - Documentation & QA

Only **ONE FINAL SYSTEM per class**

SCENARIOS:

GROUP 1: Student Enrollment System

Design an XML system to manage student enrollments.

Requirements:

- Store:
  - Student ID, Name, Course, Year Level
  - Subjects Enrolled
  - Units and Grades

Tasks:

- Create **DTD** to define structure
- Create **XSD** to enforce:
  - Valid year levels (1–4 only)
  - Units must be numeric
- Use **XPath**:
  - Retrieve all students with GPA > 2.0
- Use **XSLT**:
  - Transform into an **HTML grade report**

Self-Study:

- Use **XQuery** to list students per course
- Apply **XSD Assertion**: GPA must not exceed 5.0

****

GROUP 2: UNIFIED SYSTEM STRUCTURE (OVERALL INTEGRATION)

Main XML Root:

```
<sarms>
  <students>...</students>
  <faculty>...</faculty>
  <library>...</library>
  <billing>...</billing>
  <events>...</events>
</sarms>
```

1. One Unified XML File

- Filename: sarms.xml
- All groups must **merge outputs into ONE file**
- Use consistent naming conventions

2. One DTD and One XSD

- sarms.dtd
- sarms.xsd
- Must validate **ALL modules**

3. Modular Structure

Each group handles only their section:

<students>...</students> ← **Group 1**
<faculty>...</faculty> ← **Group 3**
<library>...</library> ← **Group 4**
<billing>...</billing> ← **Group 5**
<events>...</events> ← **Group 6**

4. One XSLT Output (Unified Report)

- Filename: sarms.xsl
- Must display:
  - Student records
  - Faculty workload
  - Library data
  - Billing summary
  - Event participation
- Output: **Dashboard-style HTML**

****

GROUP 3: Faculty Workload System

Manage faculty teaching loads.

Requirements:

- Store:
  - Faculty ID, Name, Department
  - Subjects handled
  - Teaching hours

Tasks:

- DTD for structure
- XSD:
  - Max teaching hours = 30/week
- XPath:
  - Retrieve faculty teaching more than 3 subjects
- XSLT:
  - Generate faculty workload summary

Self-Study:

- XQuery: Find overloaded faculty
- XSD Assertion: Total hours must not exceed limit

****

GROUP 4: Library Management System

Track books and borrowing records.

Requirements:

- Store:
  - Book ID, Title, Author, Category
  - Borrower info
  - Due dates

Tasks:

- DTD for book structure
- XSD:
  - Due date must be later than borrow date
- XPath:
  - List all overdue books
- XSLT:
  - Generate overdue notices (HTML)

Self-Study:

- XQuery: Retrieve most borrowed books
- XSD Assertion: Borrow date < Due date

****

GROUP 5 Scenario: Student Billing System

Manage tuition and payments.

Requirements:

- Store:
  - Student info
  - Tuition fee
  - Payments made
  - Balance

Tasks:

- DTD for structure
- XSD:
  - Payment must be ≥ 0
- XPath:
  - Retrieve students with unpaid balances
- XSLT:
  - Generate billing statements

Self-Study:

- XQuery: Calculate total revenue
- XSD Assertion: Balance = Tuition – Payments

****

GROUP 6: Event Management System

Manage university events and participants.

Requirements:

- Store:
  - Event name, date, venue
  - Participants
  - Registration status

Tasks:

- DTD for structure
- XSD:
  - Event date must not be in the past
- XPath:
  - List participants per event
- XSLT:
  - Generate event attendance sheet

Self-Study:

- XQuery: Count participants per event
- XSD Assertion: Participant must be registered before event date

DELIVERABLES

Each group must submit:

1. XML File
2. DTD File
3. XSD File
4. XPath Queries (minimum of 5)
5. XSLT File + Output (HTML)
6. Documentation:
    - System Overview
    - Explanation of XML Structure
    - Validation Rules (DTD vs XSD)
    - XPath Explanation
    - Reflection on XQuery & XSD Assertions

INSTRUCTIONS

- Work collaboratively within your group
- Ensure **well-formed and valid XML documents**
- Follow proper naming conventions
- Include comments in your code
- Submit on or before the deadline

FOR AN ADDITONAL SCORE

- Integrate all scenarios into **one unified XML system**
- Create a **dashboard-style HTML output**
- Use **multiple XSLT templates**
- Apply **complex XPath expressions**